```
TEST(FooTest, FooAddTest) {
  EXPECT_EQ(2, Foo(1, 1));
```

展开

编译：

```bash
root@65147725a2cb:/opt/test_gtest# g++ test.cc -l gtest -lgmock -lgtest_main -lpthread
```

Stub API 源码地址
将stub源码clone 到本地一个位置

```bash
git clone git@github.com:coolxv/cpp-stub.git
```

cpp-stub-ext
cpp-stub-ext克隆到本地（Optional ）

```bash
git clone git@gitee.com:master-roc/cpp-stub-ext.git
```

结合方法：

> （1）将cpp-stub-ext与cpp-stub一起导入工
> （2）修改cpp-stub的stub.h文件，将Stub类的私用函数和变量修改为保护型。

三、Stub cpp使用

1.使用教程

链接：doc_zh.md

例子大多来源于：link

1.1 不能打桩

- 不能对 exit 函数打桩,编译器做优化了
- 不能对纯虚函数打桩, 纯虚函数没有地址
- 不能对 lambda 函数打桩, lambda 函数获取不到地址
- 不能对静态函数打桩, 静态函数地址不可见.(但可以尝试使用 addr_any.h 接口获取地址)

1.2 单元测试编译选项, linux g++可用的

- -fno-access-control，private->public
- -fno-inline，禁用内联函数
- -Wno-pmf-conversions，允许成员函数转为普通函数指针
- -Wl,–allow-multiple-definition，允许重复定义
- -no-pie -fno-stack-protector，禁用栈保护（Stack Protector）功能
- -fprofile-arcs，用于生成代码覆盖率信息。
- -ftest-coverage，编译和链接时启用代码覆盖率分析。

1.3 代码覆盖率, linux g++使用方法

```bash
lcov -d build/ -z
lcov -d build/ -b ../../src1 --no-external -rc lcov_branch_coverage=1 -t ut -c -o ut_1.info
lcov -d build/ -b ../../src2 --no-extern
```

喜欢打篮球的普通人 关注 9 33 2 分享 打赏

```
lcov -a ut_1.info -a ut_2.info -o ut.info
genhtml -o report/ --prefix=`pwd` --branch-coverage --function-coverage ut.info
```

1.4普通函数打桩（非static）

cpp AI写代码

```cpp
// for linux and windows
#include "stub.h"
#include <iostream>

using namespace std;
int foo(int a) {
  cout << "I am foo" << endl;
  return 0;
}
```

展开

编译以及运行

bash AI写代码

```bash
root@65147725a2cb:/opt/test_gtest# g++ test1.cc -I ../cpp-stub/src/
root@65147725a2cb:/opt/test_gtest# ./a.out
I am foo_stub
```

1.5 对象成员函数打桩

eg1

cpp AI写代码

```cpp
// for linux, __cdecl
#include "stub.h"
#include <iostream>
using namespace std;
class A {
  int i;

public:
  int foo(int a) {
```

展开

编译以及运行

bash AI写代码

```bash
root@65147725a2cb:/opt/test_gtest# g++ test1.cc -I ../cpp-stub/src/
root@65147725a2cb:/opt/test_gtest# ./a.out
I am foo_stub
```

eg2

cpp AI写代码

```cpp
// for windows, __thiscall
#include "stub.h"
#include <iostream>
using namespace std;
class A {
  int i;

public:
  int foo(int a) {
```

展开

编译以及运行

cpp AI写代码

```bash
root@65147725a2cb:/opt/test_gtest# g++ te
root@65147725a2cb:/opt/test_gtest# ./a.ou
```

喜欢打篮球的普通人 关注 9 33 2 分享 打赏

```
I am foo_stub
```

1.6 静态成员函数打桩

cpp AI写代码

```cpp
// for linux and windows
#include "stub.h"
#include <iostream>
using namespace std;
class A {
  int i;

public:
  static int foo(int a) {
```

展开

编译以及运行

bash AI写代码

```bash
root@65147725a2cb:/opt/test_gtest# g++ test1.cc -I ../cpp-stub/src/
root@65147725a2cb:/opt/test_gtest# ./a.out
I am foo_stub
```

1.7模板函数打桩

cpp AI写代码

```cpp
// for linux, __cdecl
#include "stub.h"
#include <iostream>
using namespace std;
class A {
public:
  template <typename T> int foo(T a) {
    cout << "I am A_foo" << endl;
    return 0;
```

展开

编译及运行

bash AI写代码

```bash
root@65147725a2cb:/opt/test_gtest# g++ test1.cc -I ../cpp-stub/src/
root@65147725a2cb:/opt/test_gtest# ./a.out
I am foo_stub
```

cpp AI写代码

```cpp
// for windows, __thiscall
#include "stub.h"
#include <iostream>
using namespace std;
class A {
public:
  template <typename T> int foo(T a) {
    cout << "I am A_foo" << endl;
    return 0;
```

展开

编译及运行

bash AI写代码

```bash
root@65147725a2cb:/opt/test_gtest# g++ test1.cc -I ../cpp-stub/src/
root@65147725a2cb:/opt/test_gtest# ./a.out
I am foo_stub
```

1.8重载函数打桩

喜欢打篮球的普通人 关注 9 33 2 分享 打赏

- 如果是struct是有问题的

cpp AI写代码

```cpp
// for linux, __cdecl
#include "stub.h"
#include <iostream>
using namespace std;
class A {
  int i;

public:
  int foo(int a) {
```

展开

编译及运行

bash AI写代码

```bash
root@65147725a2cb:/opt/test_gtest# g++ test1.cc -I ../cpp-stub/src/
root@65147725a2cb:/opt/test_gtest#
root@65147725a2cb:/opt/test_gtest#
root@65147725a2cb:/opt/test_gtest# ./a.out
I am foo_stub_int5
I am foo_stub_double1.1
```

1.9 虚函数 打桩

cpp AI写代码

```cpp
// for linux
#include "stub.h"
#include <iostream>
using namespace std;
class A {
public:
  virtual int foo(int a) {
    cout << "I am A_foo" << endl;
    return 0;
```

展开

编译及运行

bash AI写代码

```bash
root@65147725a2cb:/opt/test_gtest# g++ test1.cc -I ../cpp-stub/src/
test1.cc: In function 'int main()':
test1.cc:25:38: warning: converting from 'int (A::*)(int)' to 'virtual_function_type' {aka 'int (*)(A*, int)'} [-Wpmf-c
   25 |       (virtual_function_type)(&A::foo); // 获取虚函数地址
      |                                      ^
root@65147725a2cb:/opt/test_gtest# ./a.out
I am foo_stub
```

重载的虚函数

cpp AI写代码

```cpp
//for linux gcc
#include<iostream>
#include "stub.h"
using namespace std;
class A{
    int i;
public:
    virtual int foo(int a){
        cout<<"I am A foo"<<endl;
```

展开

1.10 内联函数打桩

喜欢打篮球的普通人 关注 9 33 2 分享 打赏

> //for linux
> //添加-fno-inline编译选项，禁止内联，能获取到函数地址，打桩参考上面。

1.11 第三方库私有成员函数打桩

cpp AI写代码

```cpp
// for linux
// 被测代码添加-fno-access-private编译选项，-fno-access-private 可以让你绕过访问权限检查(好像并不需要添加该编译选项，该编译选项也不存在
// 无源码的动态库或静态库无法自己编译，需要特殊技巧获取函数地址

#include "addr_pri.h"
#include "stub.h"
#include <iostream>
using namespace std;
class A {
```

展开

编译及运行

bash AI写代码

```bash
root@65147725a2cb:/opt/test_gtest# g++ test1.cc -I ../cpp-stub/src/
root@65147725a2cb:/opt/test_gtest# ./a.out
I am A_foo 1
I am A_bar 10
I am foo_stub
I am bar_stub
```

1.12可变惨函数打桩

cpp AI写代码

```cpp
#include<iostream>
#include <stdarg.h>
#include "stub.h"
using namespace std;

double average(int num, ...)
{

    va list valist;
```

展开

1.13 仿函数打桩

cpp AI写代码

```cpp
#include<iostream>
#include "stub.h"
using namespace std;


class Foo
{
public:
    void operator() (int a)
```

展开

1.14 C语言标准库函数打桩

cpp AI写代码

```cpp
#include "stub.h"
#include <cstdio>
#include <iostream>
using namespace std;
int foo(int a) {
  puts("I am foo\n");
  return 0;
}

```

喜欢打篮球的普通人 关注 9 33 2 分享 打赏

展开

1.15 默认析构函数打桩（难）

cpp AI写代码

```cpp

#include "stub.h"
#include <iostream>
using namespace std;

template <class T> void *get_dtor_addr(bool start = true) {
  // the start vairable must be true, or the compiler will optimize out.
  if (start)
    goto Start;
```

展开

1.16 默认构造函数打桩

cpp AI写代码

```cpp
//for linux
#include<iostream>
#include "stub.h"
using namespace std;


template<class T>
void * get_ctor_addr(bool start = true)
{
```

展开

1.17 使用AddrAny对lambda函数打桩和对任意函数打桩暂缺有问题

四、 cpp-stub-ext使用（使用lambda打桩，Optional）

基于cpp-stub的功能扩展模块，提供更加丰富，方便的功能。 更加高效地在单元测试中打桩。

基于Linux 系统与 gcc/g++编译器开发

- link

使用说明：

> 使用lamda函数作为打桩函数，需调专用接口StubExt::set_lamda。

lamda函数可以使用参数和无参数：

> 为常规函数打桩，lamda有参数时lamda函数的参数列表必须与被打桩的函数参数一致。
> 为成员函数打桩，lamda有参数时lamda函数的第一个参数必须为该类的指针，接着是该成员函数的参数列表。
> 为常规函数打桩，lamda无参数可直接使用。
> 为成员函数打桩，lamda无参数可直接使用。

五、Stub cpp和Gtest结合使用

cpp AI写代码

```cpp
// Test framework
#include "gmock/gmock.h"
#include "gtest/gtest.h"

// Stub
#include "stub.h"

#include <string>

```

喜欢打篮球的普通人 关注 9 33 2 分享 打赏